

# Learning Continuous Control with Proximal Policy Optimization in a Simulated Crawler

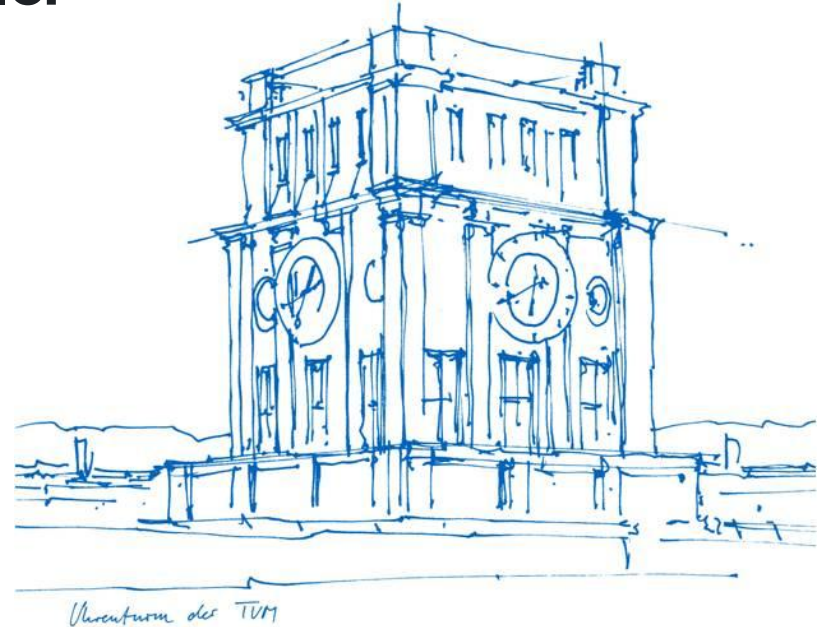
Final Project Presentation

Deep Reinforcement Learning

Technische Universität München

Group G

29 January 2026



# Problem and Motivation

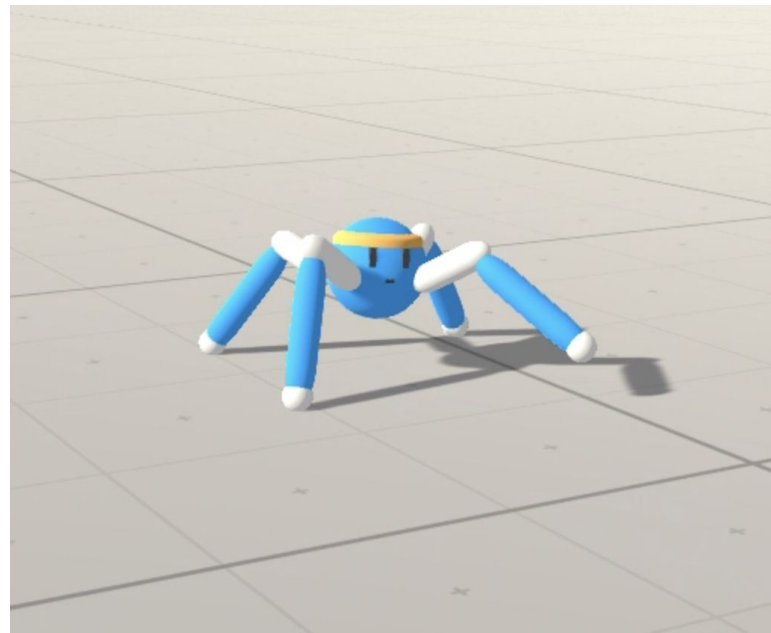
Continuous control problems are central to robotics and many real-world decision tasks.

## Problem

- Learn a **locomotion policy** for a crawler robot
- **Continuous, high-dimensional** action space
- **Long-horizon** control task with delayed effects

## Why this is challenging

- Small action changes → **large downstream behavioral effects**
- **Credit assignment** across many timesteps
- **Training instability** is common in policy learning



# Previous Work

## Previous Work on Continuous-Control Locomotion

- **Policy-gradient methods** enable continuous control, but often produce unstable or inefficient locomotion without careful task design (Williams, 1992; Konda & Tsitsiklis, 2000)
- **Trust-region and PPO-style methods** stabilize optimization, but do not directly ensure smooth or physically plausible motion (Schulman et al., 2015; Schulman et al., 2017)
- **Locomotion studies** show that reward structure strongly influences gait quality and stability (Heess et al., 2017).

**Open question:** *How much of locomotion quality comes from optimization stability versus reward design?*

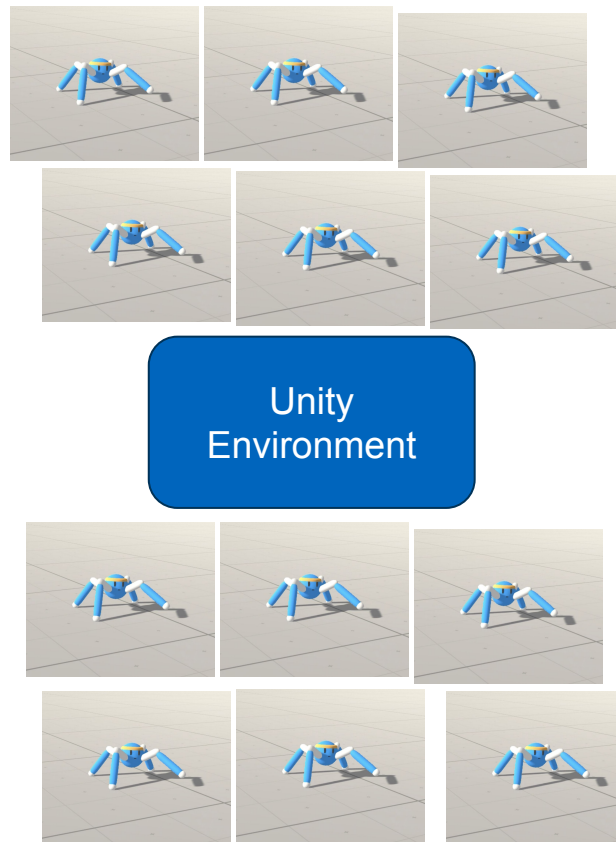


Source: Schulman et al. (2017), Proximal Policy Optimization Algorithms

# Environment and Task Description

We study the **Unity ML-Agents Crawler environment**.

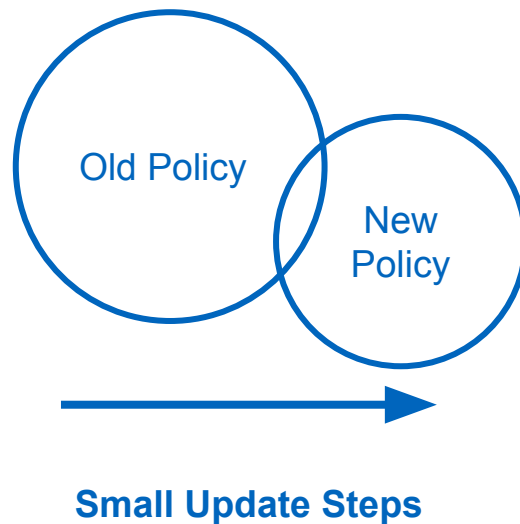
- Physics-based crawler with **4 arms and 4 forearms**
- **Goal:** move forward in a target direction without falling
- **12 parallel agents** with shared behavior parameters
- **172D state space** (joint positions, rotations, velocities)
- **20 continuous actions** (joint target rotations)
- **Geometric reward** encouraging balanced, goal-directed motion
- **Benchmark mean reward:** ~3000



# Proximal Policy Optimization (PPO)

- Policy-gradient reinforcement learning algorithm
- Optimizes a **stochastic policy**  $\pi_\theta$
- Uses **clipped objective** to ensure stable updates
- Trains **policy and value function jointly**
- Well-suited for **continuous control** tasks

Improve the policy without  
making overly large updates



# PPO Objective Function

1. Policy ratio :  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
2. Advantage function:  $\hat{A}_t$
3. Clipped surrogate objective:  $L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[ \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right]$

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t [L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)],$$

value function loss

entropy bonus

# PPO Objective Function

## Geometric (multiplicative) reward formulation

Reward computed **per time step**

Encourages satisfying **all objectives simultaneously**

Main reward components:

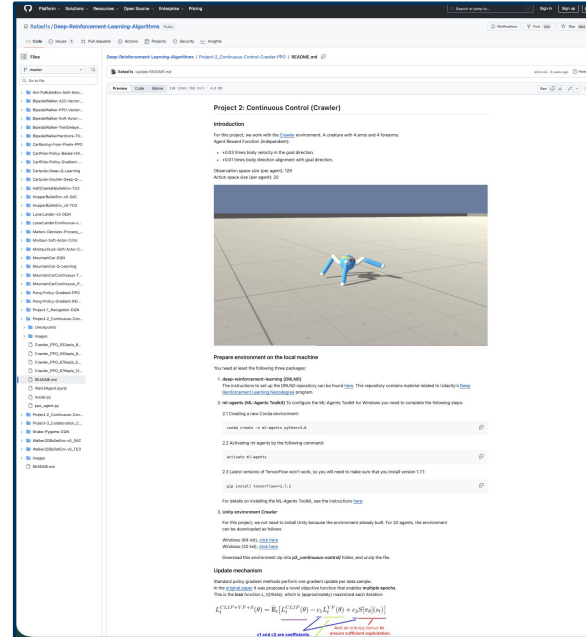
- Forward body velocity toward goal
- Head direction aligned with goal

All reward terms **normalized to [0, 1]**

$$r_t = r_{\text{velocity}}(t) \cdot r_{\text{alignment}}(t)$$

# Methodology

- Used an open-source PPO crawler implementation in Unity ML-Agents based on Schulman et al. (2017)
- Following Heess et al. (2017), we improve locomotion by shaping the reward rather than modifying PPO





$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)],$$

$$r_t^{\text{new}} = r_t^{\text{env}} - \mathcal{P}_t$$

$r_t^{\text{env}}$  : original task reward (forward motion)

$\mathcal{P}_t$  : penalizes unstable locomotion behaviors not covered by the original reward

# Final Reward Function

## Problem Observed

## Added Term

Body sway / yaw

Smoothness  $-\lambda_{smooth} \|a_t - a_{t-1}\|^2$

Asymmetric stepping

Symmetry  $-\lambda_{sym} \|a^L - a^R\|^2$

Big lateral leg swings

Magnitude  $-\lambda_{mag} \|a_t\|^2$

$$r_t^{\text{final}} = r_t^{\text{env}} - \lambda_{\text{smooth}} \|a_t - a_{t-1}\|^2 - \lambda_{\text{sym}} \|a_t^L - a_t^R\|^2 - \lambda_{\text{mag}} \|a_t\|^2$$

# Experimental Comparison Overview

Compared approaches:

- Baseline PPO (original implementation)
- PPO + reward shaping
- PPO + reward shaping + stabilization (final model)

Evaluation criteria:

- Final episodic return
- Sample efficiency (episodes to reach target score)
- Stability across random seeds



# Final Reward Function Implementation

## Added Term

Smoothness  $-\lambda_{smooth} \|a_t - a_{t-1}\|^2$

Symmetry  $-\lambda_{sym} \|a^L - a^R\|^2$

Magnitude  $-\lambda_{mag} \|a_t\|^2$

```
# ===== Smoothness penalty =====
if self.USE_SMOOTHNESS:
    diff = actions - self.prev_actions
    smooth_pen = np.sum(diff ** 2, axis=1)
    rewards_new -= self.LAMBDA_SMOOTH * smooth_pen

# ===== Symmetry penalty =====
if self.USE_SYMMETRY:
    a_dim = actions.shape[1]
    half = a_dim // 2
    left = actions[:, :half]
    right = actions[:, half:]
    sym_pen = np.sum((left - right) ** 2, axis=1)
    rewards_new -= self.LAMBDA_SYM * sym_pen

# ===== Magnitude penalty =====
if self.USE_MAGNITUDE:
    mag_pen = np.sum(actions ** 2, axis=1)
    rewards_new -= self.LAMBDA_MAG * mag_pen
```

# Quantitative Comparison (800 Avg Score)

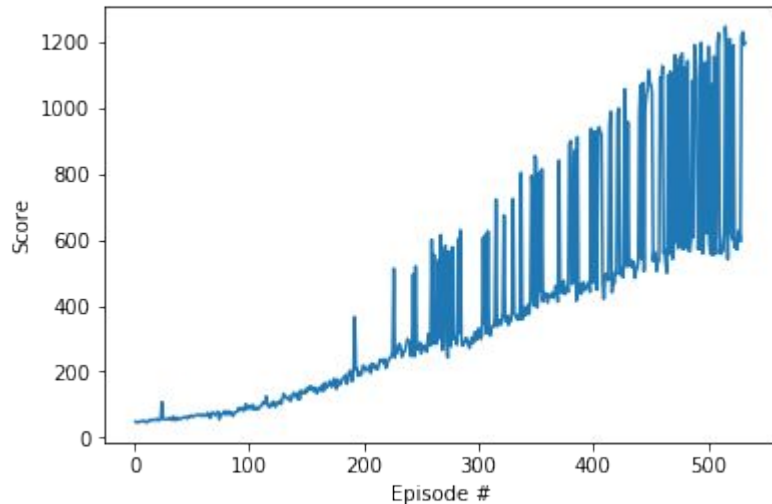
| Penalties     | Run 1 | Run 2 | Run 3 | Run 4 | Run 5 | Run 6 | Run 7 |
|---------------|-------|-------|-------|-------|-------|-------|-------|
| Smoothness    | x     |       |       | x     | x     |       | x     |
| Symmetry      |       | x     |       | x     |       | x     | x     |
| Magnitude     |       |       | x     |       | x     | x     | x     |
| # of episodes | 392   | 468   | 500   | 450   | 441   | 364   | 500   |



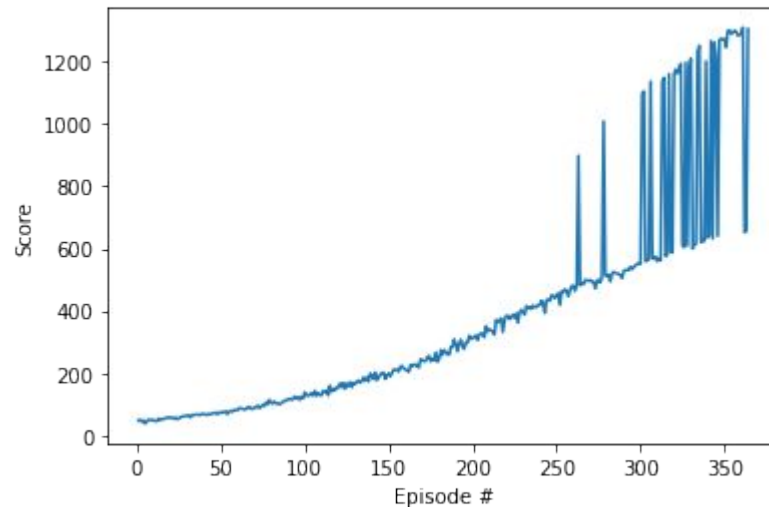
Most efficient run i.e.  
quickest to 800 score

# Scores per Episode

Original (532 episodes)



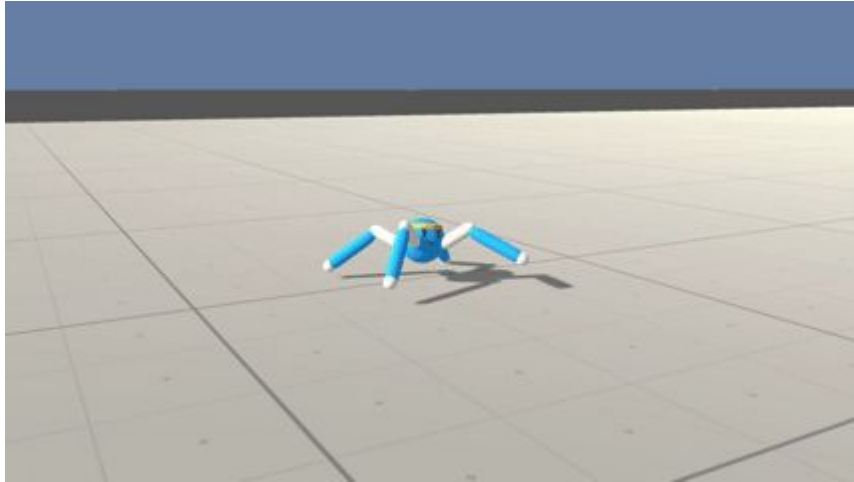
Improved (364 episodes)



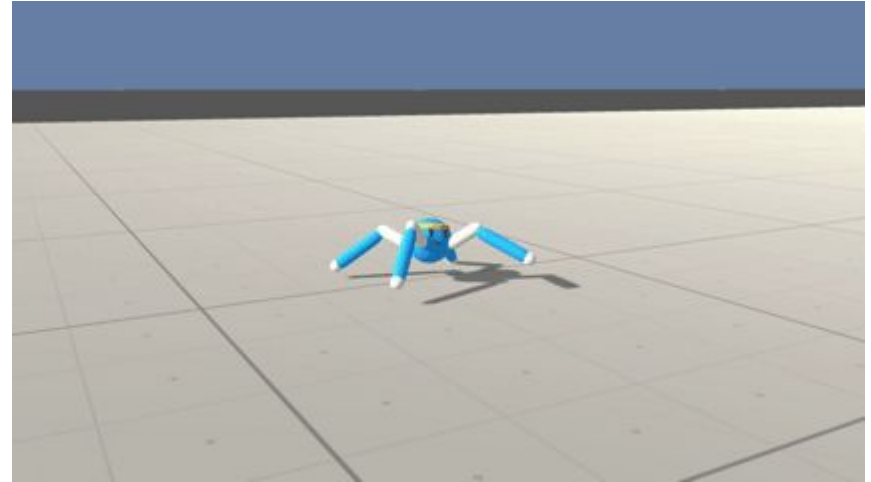
Our model reaches strong performance faster

# Comparison of Visuals (800 score)

Original (532 episodes)

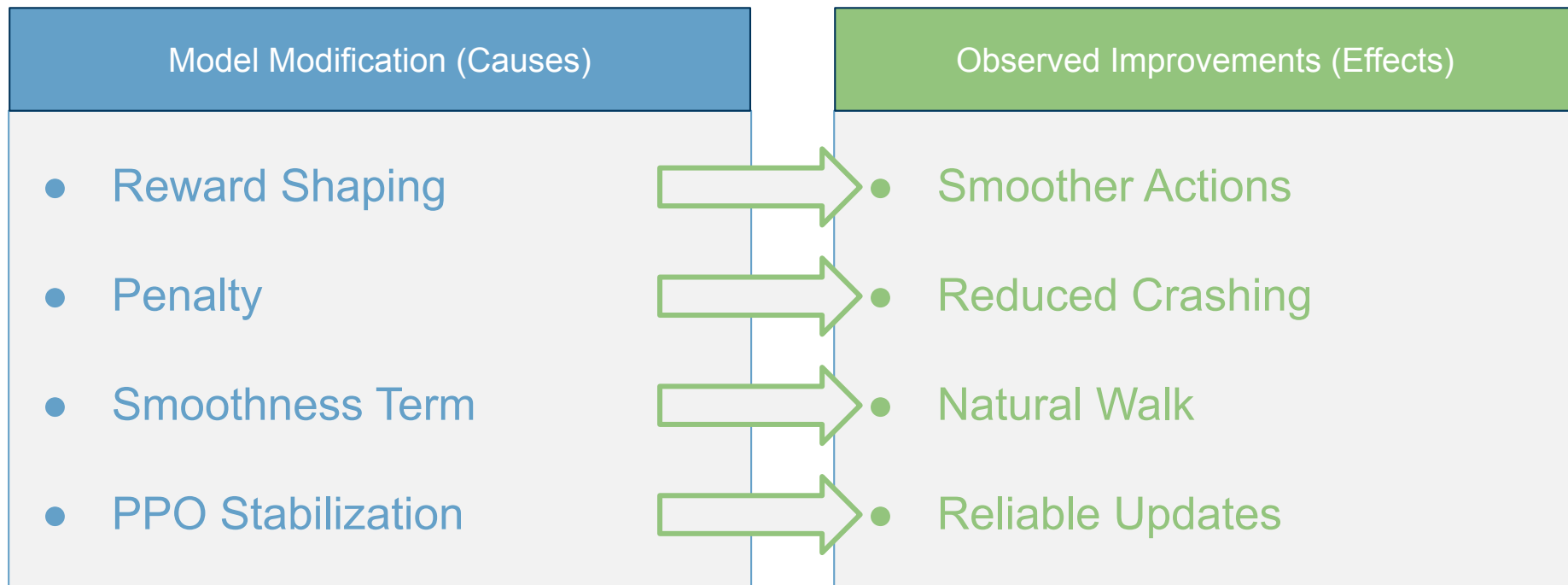


Improved (364 episodes)



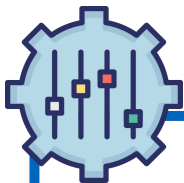
34% decrease in the # of episodes - the same score

# Why Did Our Model Improve?





# Limitations



Sensitivity to reward shaping coefficients

<sup>T H E</sup>  
MATRIX

Simulation-only evaluation



Limited hyperparameter search



Computation/budget constraints

**1. More complex locomotion environment**

**2. Domain randomization**

**3. Sim-to-real transfer**

**4. Scaling**

# Thank You ! Questions?

